CS 111

Foundations of Program Design

Week 12, Wednesday Lecture

Quiz 7

In-class, 15 minutes.

Project 2

Please submit your finished brush for use in lab next week. I'd love to see them all together.
https://forms.gle/s2fGVnBugHeFKMTa8

Variable scope

Now that we are writing sketches outside the Processing IDE, we have our own version of Processing's PApplet class.

```
public class PApplet {
    public void main(String[] args) {
        // entry point method
    }

    public void rect(float x, float y, float width, float height) {
        // ...
    }
}
```

Often, we'll need to share its methods with other classes. Why does this code report an error?

```
public class Shape {
    PVector position;

    public void draw() {
        rect(position.x, position.y, 20, 20);
        // since I'm not a PApplet, I don't know the rect method
    }
}
```

The Shape instance doesn't have access to the PApplet's fields and methods. This is an intentional design constraint in Java.

Each object instance has access to its own fields and methods (from its class definition), and doesn't know about other objects' fields and methods. Fields belonging to other classes are *outside the scope* of variables your class definition can reference.

(Scope works the same way for methods defined in classes, as it does for variables defined in control flow statements like `if` and `for`.)

Reading Java source code line-by-line, you can always see which variables are *in scope* by reading the method arguments and variable definitions from every block you're inside of.

In lab on Monday, you passed the self-reference variable `this` from a PApplet instance to a Shape instance, giving your Shape access to the PApplet's drawing methods.

```
public class Shape {
    PApplet p;
    public Shape(PApplet parent, ...) {
        // ...
        p = parent;
    }
}
```

Is this an argument or an expression?

Let's define a constructor, and observe two different ways that the same *variable reference* can be read, in its two capacities: the variable is a box for *storing values* and for *reporting values*.

```
import java.time.LocalDate;

public class SimpleTime {
    LocalDate lastModified;
    int hour, minute, second;

    public SimpleDate(int hour, int minute, int second) {
        this.hour = hour;
        this.minute = minute;
        this.second = second;
        lastModified = LocalDate.now();
    }

    @Override
    public String toString() {
        return hour + ":" + minute + ":" + second +
                " (updated " + lastModified.toString() + ")";
    }
}
```

Static methods

Let's revisit that LocalDate constructor. It wasn't really a constructor, since it wasn't using the `new` keyword, but it does behave like one.

```
lastModified = LocalDate.now();
```

LocalDate is not an object instance of the LocalDate class. Instead, `static LocalDate now()` is a static **factory method** which produces a LocalDate object.

It is common for Java libraries to define factory methods which produce an instance object. It might not be a new instance object; sometimes there are reasons to re-use existing instances.

Deep equality vs. reference equality

Recall from last week's lecture that the `boolean equals(Object that)` method is defined for all Objects. But it doesn't know anything about the fields or methods of your object; so it compares the two **object references**, and not the values your object stores.

```
PVector a = new PVector(2, 2);
PVector b = new PVector(2, 2);
a.equals(b); // true
```

Project 3

You will build a painting application with multiple animated brushes, building on your Project 2 submission, and the ability to save or load a canvas from the filesystem using a CSV table.

In order to support multiple brushes in the save-load system, you will define a Shape class that defines all state for each of its subclasses. You will also need to indicate to the user which brush they have selected, and some way of switching between brushes.

You will submit your project and supporting documentation by the start of class on Wednesday, December 2nd.

Optionally, you may add the ability to clear the canvas or to delete individual strokes. As a further stretch goal, consider adding filters, or suggesting your own new operation.